ECE 310
Project 1 Report

Ezana Enquobahrie
February 12, 2026

Abstract

The aim of this project was to design, build and test an 8x8 unsigned Dadda multiplier using Gate-level Verilog Language. The goal was to show how different architectures of multipliers can quickly compute partial products with structured reduction. The design was implemented strictly at the Gate level, meaning only AND gates were used to create the partial products and only Half Adders and Full Adders were used to reduce them according to the project's specifications. No behavioral operators or different multiplier architectures were utilized.

The Dadda Reduction Technique was applied to restrict the height of the maximum column at each reduction stage, creating a systematic and effective reduction tree. The remaining two rows of the reduction were added together using a Ripple Carry Adder to produce the final 16-bit output product of the design. The final design consisted of 64 AND gates, 48 Full Adders and 8 Half Adders. A simulation was conducted using Vivado to verify functionality by providing various test vectors. All test vectors resulted in the actual output of the Dadda multiplier matching as expected from the expected multiplied Output.

1 Introduction, Background, and Theory

1.1 Motivation for Using a Dadda Multiplier

Multiplying in binary is one of the key operations performed by any digital system; however, simple implementations (like the shift-and-add method) can result in inefficient designs due to long carry propagation paths. To enhance system performance, tree-based multiplier designs can produce many partial products through a parallel operation.

In terms of tree multipliers, Dadda multipliers are designed to balance speed and hardware resources by minimizing the height of the column of numbers that make up the partial products but only doing this when absolutely necessary.

1.2 Partial Products and Bit Weights

Each partial product is formed as $P_{ij}=a[i] \wedge b[j]$ and belongs to a column with weight $i+j$. Correct handling of these weights is critical, since all inputs to a Half Adder or Full Adder must originate from the same column. The sum output remains in the same column, while the carry output propagates to the next higher-weight column.

1.3 Gate-Level Design Requirement

This project required strict gate-level modeling. Partial products were generated explicitly using AND gates, and Half Adders and Full Adders were implemented using only XOR, AND, and OR gates. Behavioral operators such as * or + were not used in the multiplier design itself. This constraint ensures that the design reflects actual hardware structure rather than abstract behavior.

2 Results and Analysis

2.1 Design Structure and Hardware Counts

The final multiplier design consists of three main components:

1. Partial product generation using 64 AND gates
2. A Dadda reduction tree using 48 Full Adders and 8 Half Adders
3. A final ripple-carry adder to produce the 16-bit output

The hardware counts exactly match the expected values for an 8×8 Dadda multiplier and were verified by the autograder.

2.2 Testbench Methodology

A custom Verilog testbench was created to verify functional correctness. The testbench applies input vectors to the multiplier and computes an expected result using behavioral multiplication. The output of the multiplier (prod) is then compared against this expected value.

2.3 Selection of Test Vectors

The test vectors were chosen to exercise different aspects of the multiplier rather than relying on trivial cases:

- **Small non-trivial operands (e.g., 0x0D × 0x11):**
  These verify basic correctness while involving multiple partial products across different columns.

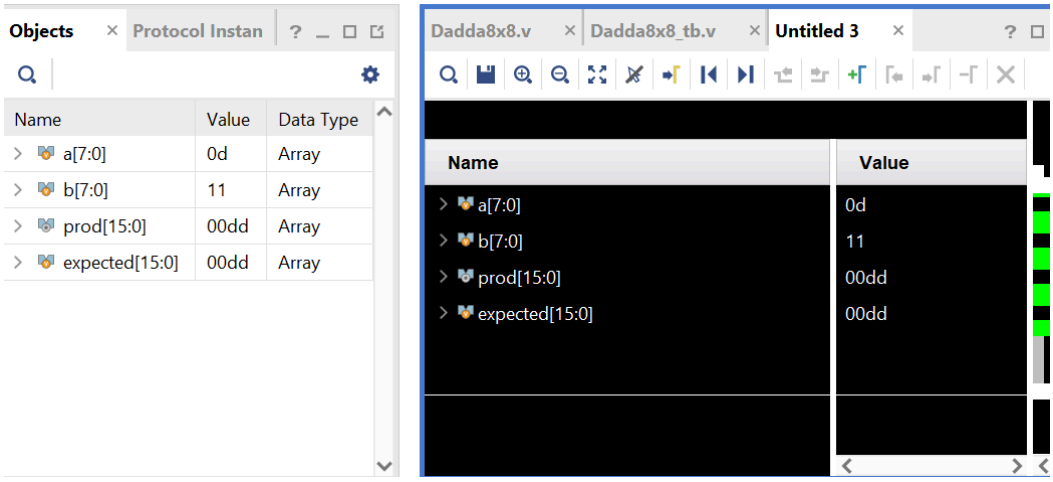


Figure 1:0x0D × 0x11

- **Mixed mid-range values (e.g., 0x3C × 0xA7):**
  These produce a dense partial product matrix and multiple internal carries, stressing the reduction tree.

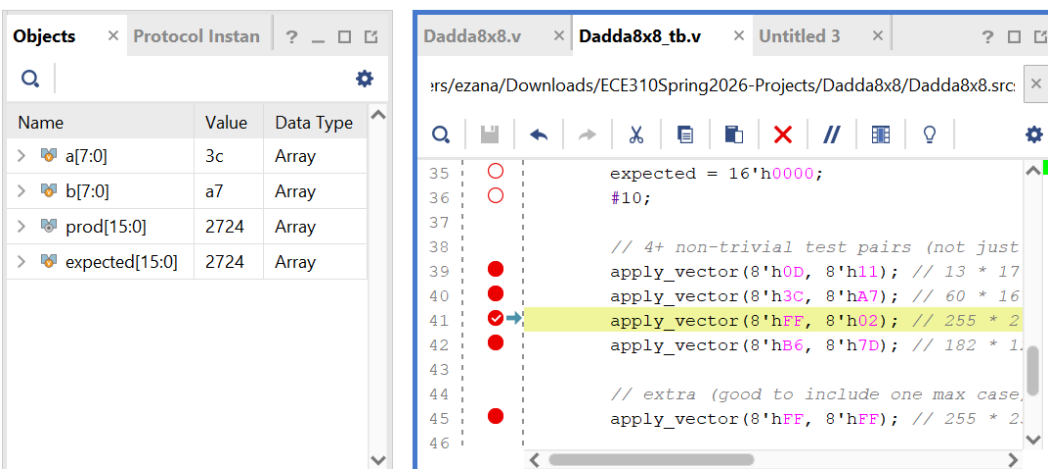


Figure 2:0x3C × 0xA7

- **Edge-style case (e.g., 0xFF × 0x02):**
  This case checks for correct bit ordering and verifies that shifting behavior emerges naturally from the reduction logic.

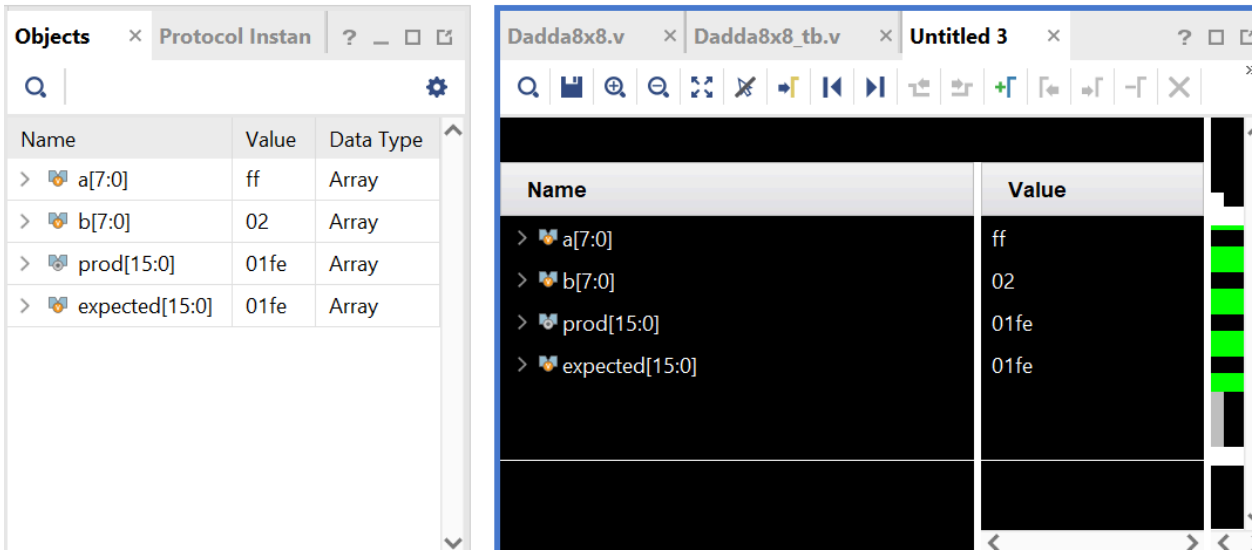


Figure 3:0xFF× 0x02

- **Carry-heavy case (e.g., 0xB6 × 0x7D):**
  This stresses both the reduction tree and the final ripple-carry adder, ensuring correct carry propagation across many columns.

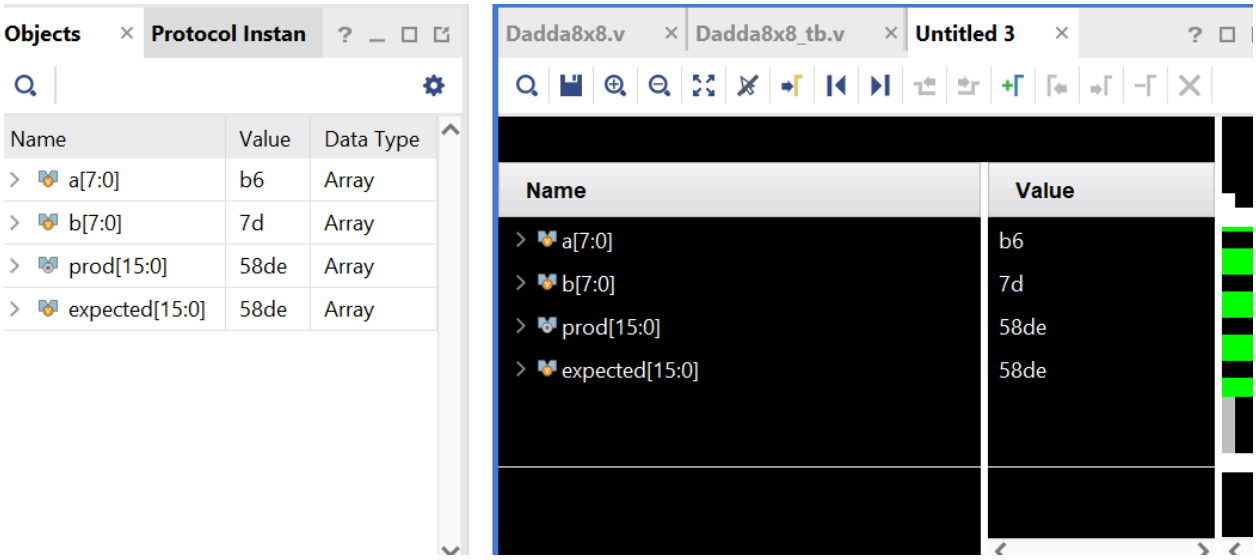


Figure 4:0xB6× 0x7D

2.4 Simulation Results

Simulation results showed that, for all test vectors, the output prod matched the expected multiplication result exactly. Waveforms confirm the correct propagation of sums and carries

throughout the design. Figures included in the report show the input operands, the resulting product, and the expected value for each test case, demonstrating functional correctness.

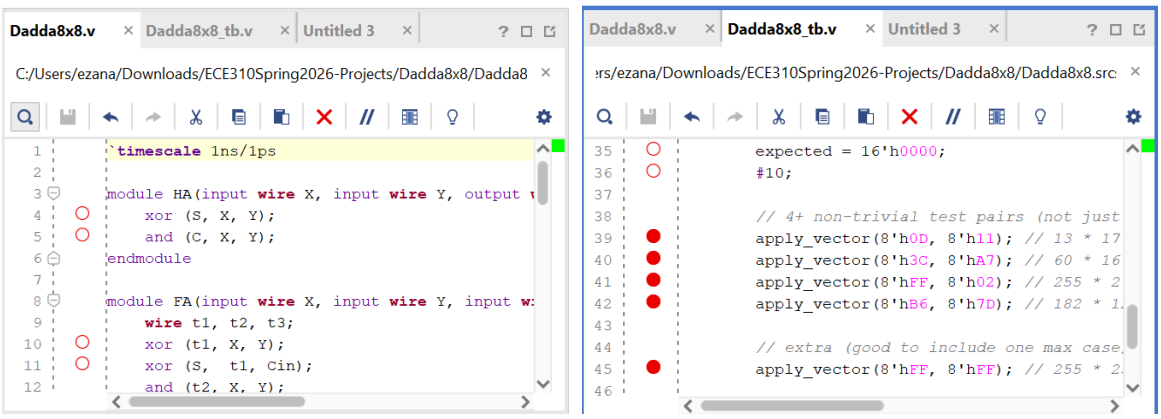


Figure 5: Vivado Sources window showing separate files for the Dadda8x8 model (Dadda8x8.v) and the testbench (Dadda8x8_tb.v).

The design and testbench were implemented in two separate Verilog files, as required.

3 Conclusions

The 8×8 unsigned Dadda multiplier was successfully designed and verified using gate-level Verilog modeling. By following the Dadda reduction methodology, partial products were efficiently reduced while minimizing the number of adders used. Careful attention to bit weights and carry propagation ensured correctness throughout the design.

Simulation results confirmed that the multiplier produced correct results for multiple non-trivial input combinations. The final design met all project constraints, passed all autograder structural and functional tests, and demonstrated a clear understanding of fast multiplier architectures and gate-level hardware design principles.

Verilog Implementation Overview:

1. Half Adder and Full Adder Modules

```
module HA(input wire X, input wire Y, output wire S, output wire C);
   xor (S, X, Y);
   and (C, X, Y);
endmodule

module FA(input wire X, input wire Y, input wire Cin, output wire S, output wire C);
   wire t1, t2, t3;
   xor (t1, X, Y);
   xor (S,  t1, Cin);
   and (t2, X, Y);
   and (t3, t1, Cin);
   or  (C, t2, t3);
endmodule
```

2. Partial Product Generation Example

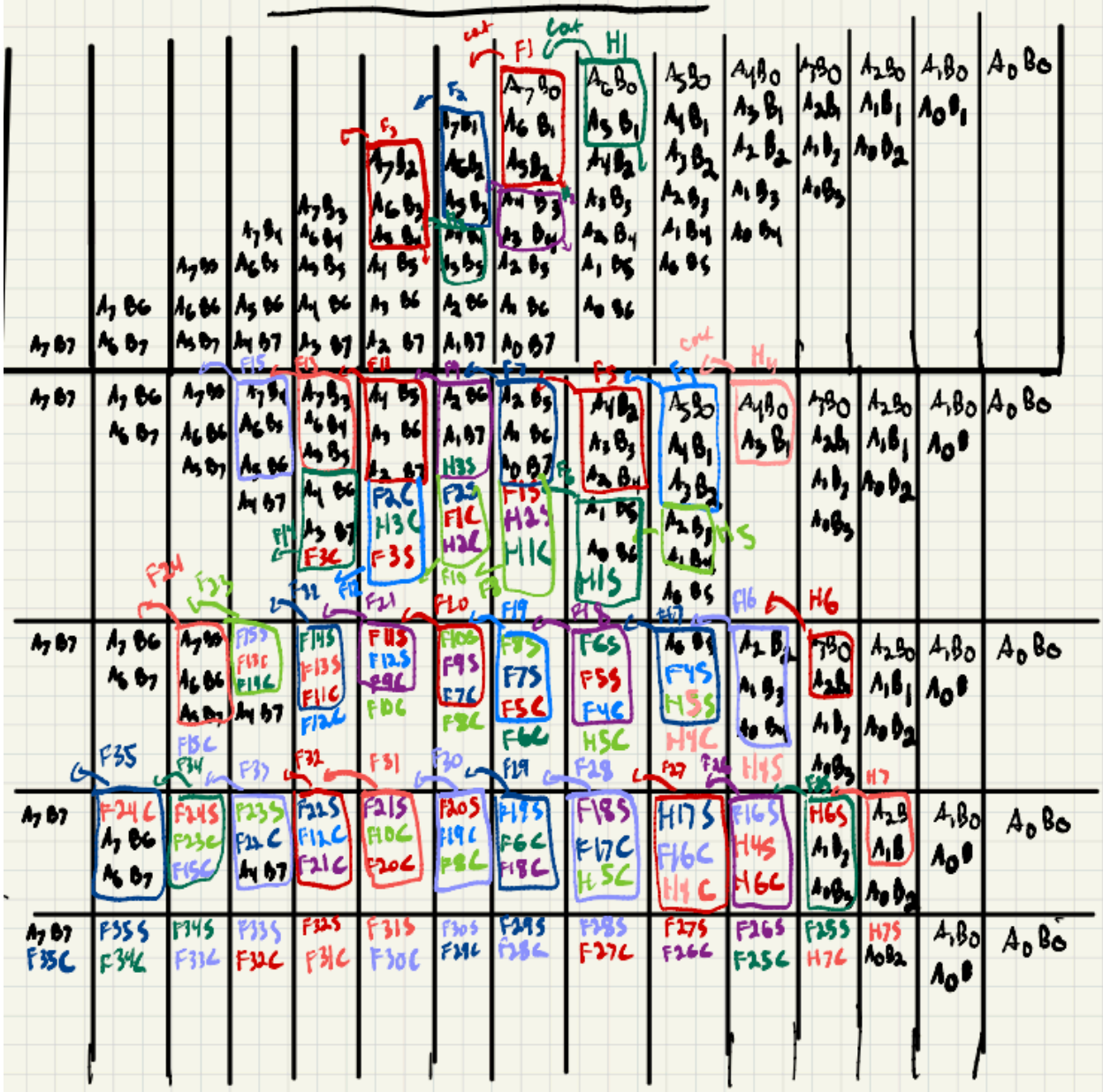
```
and (P00, a[0], b[0]);
and (P10, a[1], b[0]);
and (P01, a[0], b[1]);
```

3. Reduction Stage Example

```
HA H1 (.X(P60), .Y(P51), .S(H1S), .C(H1C));
FA F1 (.X(P70), .Y(P61), .Cin(P52), .S(F1S), .C(F1C));
```

Used design for project1:

$A_7\ A_6\ A_5\ A_4\ A_3\ A_2\ A_1\ A_0$ ×

$B_7\ B_6\ B_5\ B_4\ B_3\ B_2\ B_1\ B_0$

7 Half Adders
35 Full Adders
64 AND gates